

Carreiras críticas no problema do produtor-consumidor

Nuria Guerra Casal Alejandra Linaje Vallejo
Grao en Enxeñaría Informática

1. Introducción

O problema do produtor-consumidor é un problema de sincronización en sistemas operativos: un ou varios fíos produtores xeran datos, e deposítanos nun buffer compartido, mentres que un ou varios fíos consumidores extráenos e procesanos. Cando este acceso concorrente ao buffer faise sen ningunha sincronización, prodúcese **carreiras críticas**: situacións nas que o resultado final do programa depende da orde imprevisible na que o sistema operativo planifica a execución de cada fío, o que da lugar a resultados incorrectos e non reproducibles.

Neste apartado demóstrase de forma práctica a existencia das carreiras críticas mediante dous fíos (produtor e consumidor), que comparten un buffer FIFO de enteiros **sen ningunha protección**. Para aumentar a probabilidade de que se manifesten as carreiras, introdúcense `sleep()`s estratéxicamente situados nas rexións críticas, ampliando a ventá temporal na que os dous fíos poden entrar en conflito.

2. Estrutura do código

O programa `prod_cons.c` implementa ambos fíos mediante `pthread`. Ao usar fíos nun mesmo proceso, a memoria é compartida directamente sen necesidade da función `mmap()`.

2.1. Buffer FIFO compartido

O buffer é un array circular de enteiros de tamaño `TAMANO_BUFFER=10`, representado pola seguinte estrutura:

```
1 typedef struct {
2     int buf[TAMANO_BUFFER]; //array circular de enteiros
3     int cabeza; //índice de saída: remove_item le de aquí
4     int cola_fin; //índice de entrada: insert_item escribe aquí
5     int conta; //numero de elementos no buffer neste momento
6     int rematou; // o produtor acabou de ler o ficheiro
7 } BufferFIFO;
```

Listing 1: Estrutura do buffer FIFO compartido

Os tres campos `cabeza`, `cola_fin` e `conta` son os recursos compartidos críticos: tanto o produtor como o consumidor len e modifican estes datos sen ningún control, o que é precisamente o que provoca as carreiras.

Ademais, mantéñense dúas variables globais, `suma_produtor` e `suma_consumidor`, que acumulan respectivamente os valores lidos do ficheiro e os valores consumidos. A diferenza entre elas ao final da execución é a proba de que ocorreron carreiras críticas.

2.2. Funcións principais

- `produce_item(f)`: le un enteiro do ficheiro de texto e súmao a `suma_productor`.
- `insert_item(numero)`: insire o enteiro no buffer sen sincronización.
- `remove_item()`: retira un enteiro do buffer sen sincronización.
- `consume_item(numero)`: suma o enteiro a `suma_consumidor`.

3. Mecanismo de inxección das carreiras críticas

O patrón que provoca as carreiras é idéntico en `insert_item()` e en `remove_item()`: primeiro lese e gárdase localmente o estado do buffer, despois dórmese 2 segundos (momento no que o outro fío pode modificar os valores, ocasionando a carreira crítica), e finalmente escíbese co valor local xa obsoleto, ignorando calquera cambio que o outro fío puidese facer mentres tanto.

```
1 // Gardase o estado actual do buffer localmente
2 int pos_vella    = buffer.colafin;
3 int conta_vella = buffer.conta;
4
5 sleep(2); // O consumidor pode alterar 'cabeza' e 'conta', provocando unha
           // carreira crítica
6
7 // Escribese cos índices gardados, ignorando os cambios do consumidor
8 buffer.buf[pos_vella] = numero;
9 buffer.colafin = (pos_vella + 1) % TAMANO_BUFFER;
10 buffer.conta = conta_vella + 1; // Sobreescríbe o valor real de conta
```

Listing 2: Carreira crítica en `insert_item()` – o sleep amplía a ventá de conflito

```
1 int pos_vella    = buffer.cabeza;
2 int conta_vella = buffer.conta;
3
4 sleep(2); // O produtor pode alterar 'colafin' e 'conta', provocando unha
           // carreira crítica
5
6 int numero = buffer.buf[pos_vella];
7 buffer.buf[pos_vella] = 0;
8 buffer.cabeza = (pos_vella + 1) % TAMANO_BUFFER;
9 buffer.conta = conta_vella - 1; // Sobreescríbe o valor real de conta
```

Listing 3: Carreira crítica en `remove_item()` – patrón simétrico

O `sleep(2)` entre a lectura e a escritura garante que, con alta probabilidade, o outro fío execute a súa parte central durante ese intervalo. Cando ambos fíos len `conta` antes dos seus respectivos `sleep()`, cada un traballa cunha copia local do valor, que xa non é válida cando volven escribir.

3.1. Finalización do consumidor

O consumidor executa un bucle no que:

- Se o buffer está baleiro e `rematou == 0`, espera a que o produtor xere novos datos.
- Se o buffer está baleiro e `rematou == 1`, remata a execución.
- Se hai datos no buffer, retíraos e consúmeos normalmente.

Este mecanismo evita que o consumidor remate antes de tempo e garante que procesa todos os elementos dispoñibles, salvo os efectos derivados das carreiras críticas.

4. Execución

```
# Compilar
gcc -o prod_cons prod_cons.c -pthread

# Executar cun ficheiro
./prod_cons numeros.txt
```

O ficheiro de entrada `numeros.txt` debe conter números enteiros positivos de dúas cifras (entre 1 e 99) separados por espazos en branco. O fío produtor produce ata que remate de ler o ficheiro.

5. Observación das carreiras críticas

5.1. Escenario típico de carreira

A seguinte secuencia amosa como se produce a carreira crítica cando ambos fíos acceden case simultaneamente:

1. O **produtor** le `conta=1` e garda `conta_vella=1`, `pos_vella=5`. Dorme 2s.
2. O **consumidor** le `conta=1` e garda `conta_vella=1`, `pos_vella=7`. Dorme 2s.
3. O **consumidor** esperta: retira o valor de `buf[7]` e pon `conta=0`.
4. O **produtor** esperta: insire en `buf[5]` e pon `conta = 1+1 = 2`, ignorando que xa valía 0.

O resultado é que `conta` pasa de 0 a 2 sen pasar por 1, e o consumidor pode ler posicións que contén 0 (“baleiro”) en lugar de datos válidos, o que distorsiona a suma final.

5.2. Evidencia nos resultados

A proba definitiva das carreiras críticas é que, ao rematar ambos fíos, a suma do produtor e a suma do consumidor **non coinciden**. Na execución de referencia obtívose:

```
Suma do Produtor    : 5421
Suma do Consumidor  : 4133
```

Figura 1: Saída por terminal da suma alcanzada.

A diferenza débese a que: ou ben, o consumidor leu algunhas posicións que contiñan 0 en lugar de datos reais (porque o produtor aínda non escribira nelas), ou algúns valores foron sobreescritos antes de ser consumidos. Como se pode apreciar na saída por terminal, aparecen liñas como [Consumidor] Consume: 0, que constatan a lectura de posicións baleiras.

```
[Produtor]: ve conta=0, garda local=0. Destino buf[9]
[Consumidor]: Retira 0 de buf[9]. Nova conta=0 (ignorando cambios)
[Consumidor] Buffer: [ 57 36 92 64 49 81 20 75 43  0 ] cabeza=0 cola_fin=9

[Consumidor] Consume: 0
[Produtor]: Inserta 11 en buf[9]. Nova conta=1 (ignorando cambios)
[Produtor]   Buffer: [ 57 36 92 64 49 81 20 75 43 11 ] cabeza=0 cola_fin=0
```

Figura 2: Saída por terminal mostrando carreiras críticas, consume 0.

Tamén se observan saltos anómalos en `conta`: pasa de valores como 0 a 2 ou de 1 a 3 nun só paso, evidenciando que os cambios de ambos fíos solápanse e machácanse mutuamente.

```
[Produtor]: ve conta=1, garda local=1. Destino buf[8]
[Consumidor]: ve conta=1, garda local=1. Orixe buf[2]
[Consumidor]: Retira 72 de buf[2]. Nova conta=0 (ignorando cambios)
[Consumidor] Buffer: [ 0 0 0 15 84 41 68 23 55 0 ] cabeza=3 cola_fin=9

[Consumidor] Consume: 72
[Produtor]: Inserta 55 en buf[8]. Nova conta=2 (ignorando cambios)
[Produtor] Buffer: [ 0 0 0 15 84 41 68 23 55 0 ] cabeza=3 cola_fin=9
```

Figura 3: Saída por terminal mostrando carreiras críticas, saltos na conta.

6. Conclusións

Este apartado demostra de forma experimental que o acceso concorrente a variables compartidas sen sincronización produce resultados incorrectos e non deterministas. As tres variables do buffer (`cabeza`, `cola_fin`, `conta`) son críticos: calquera lectura, modificación ou escritura non atómica sobre elas pode deixar o buffer nun estado inconsistente.

Os efectos observados son: o consumidor consome ceros en lugar de datos reais, `conta` realiza saltos imposibles, e as sumas finais do produtor e consumidor diverxen. A solución a este problema, mediante mutex e variabeis de condición, realízase no seguinte apartado.